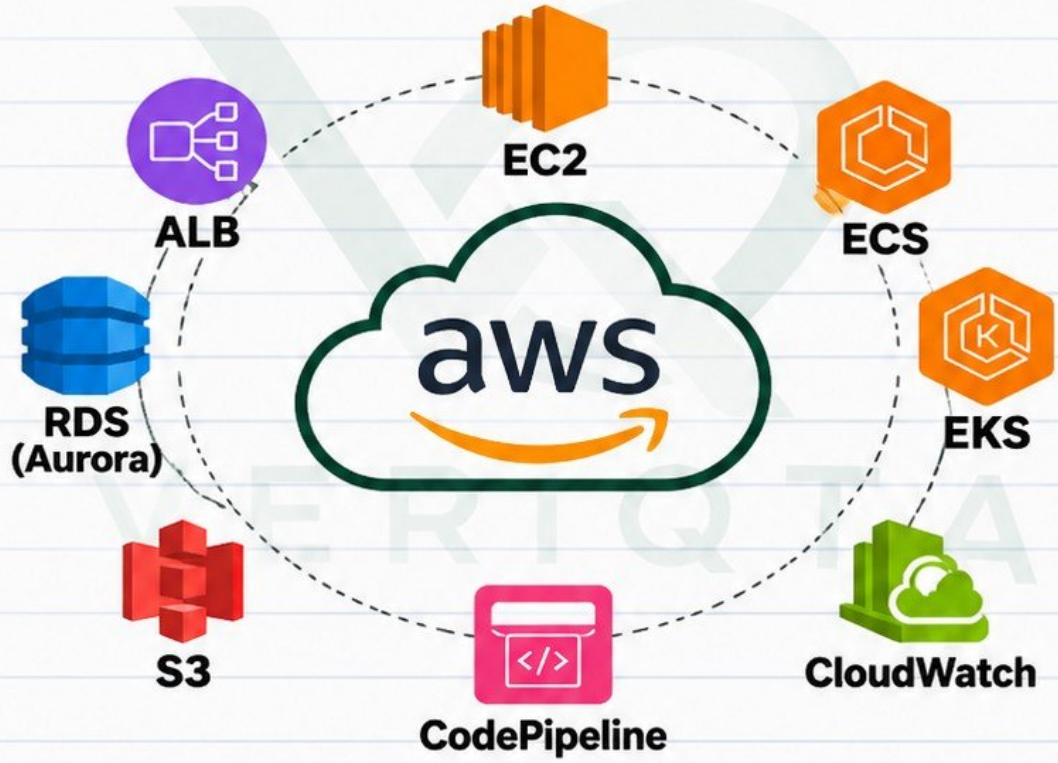
VERIQTA

10 AWS Projects That Prove You Can Do the Job



Real-World, Production-Ready AWS Projects

Architecture Build | Secure Operate | Scale Recover

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

Project 1: Survive an Availability Zone Failure Without Losing Orders

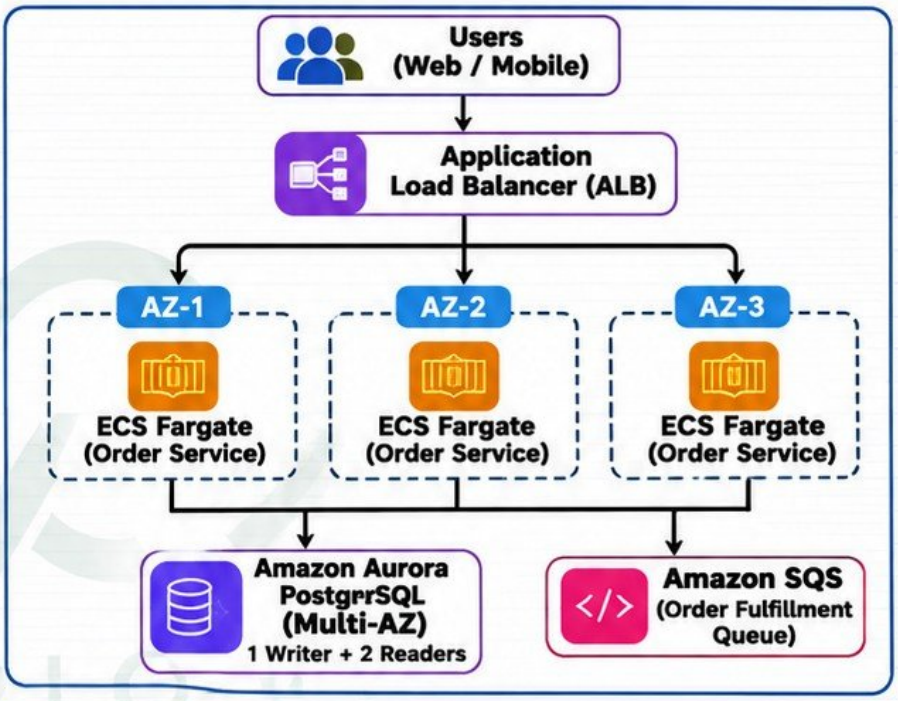
Design an Order Processing System That Survives an AZ Outage

Objective

- Build a highly available ordering platform that continues to accept and process orders even if one Availability Zone (AZ) becomes unavailable.

Architecture (Multi-AZ)

- **ALB:** Distributes traffic across healthy AZs.
- **ECS Fargate:** Runs stateless order services in 3 AZs.
- **Aurora PostgreSQL:** Multi-AZ (1 writer, 2 readers).
- **SQS:** Reliable, decoupled order fulfillment.



Key Design Decisions

Failure Domains	AZ is the blast radius; design for loss of one AZ.
DB Writer Placement	Single writer in one AZ with automatic failover to another.
Idempotency Keys	Unique key (e.g., customer + request ID) to prevent duplicate orders during retries.
Transaction Boundaries	Keep the "accept order" transaction small and fast; do not include fulfillment in the same transaction.

Accept vs. Fulfill

Accepting an Order

- Validates, stores the order, and returns a confirmation immediately.
- Must be fast, highly available, and tolerant to AZ failures.

Completing Fulfillment

- Asynchronous work (e.g., payment capture, inventory update, shipment).
- Runs via SQS workers and can retry safely without blocking orders.

Senior Engineer Insight

Design for graceful degradation: Orders must still be accepted if fulfillment or a single AZ is down. Use idempotency, decoupling (SQS), and multi-AZ persistence to prevent data loss and duplicates.

Benefits

- Survives an AZ outage
- No duplicate orders
- Fast order acceptance
- Scales and remains reliable

Project 1: Survive an Availability Zone Failure Without Losing Orders

Kill an Availability Zone and Prove the System Recovers

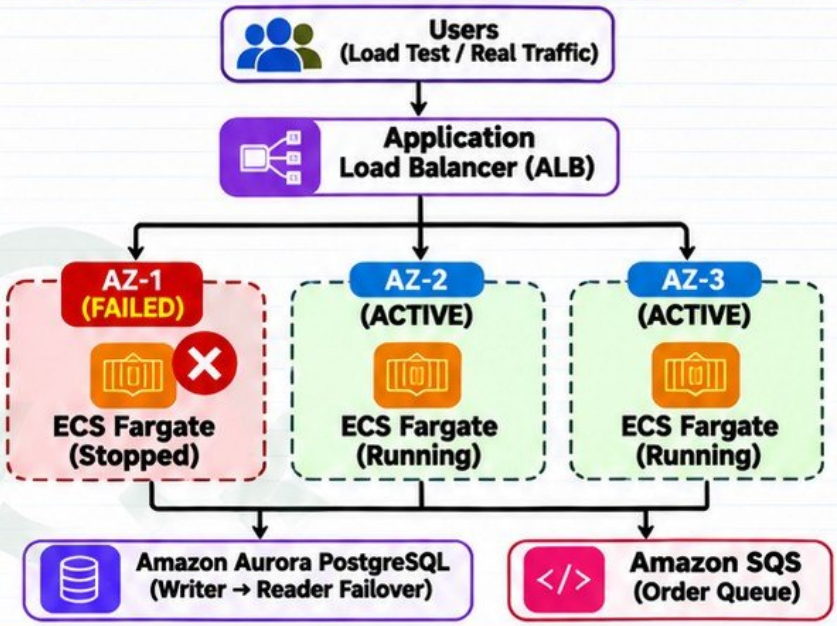
Test Scenarios

1. **AZ Failure (App Layer)**
   - Remove ECS capacity in AZ-1 and observe recovery.
2. **DB Writer Failover**
   - Trigger an Aurora writer failover and validate application continuity.

What to Investigate

- Connection resets to DB.
- Retry storms (client and app).
- Duplicate order risks.
- SQS queue backlog and drain.
- Recovery time (RTO).
- Service availability and errors.

Test Setup: Multi-AZ Order Platform



Measure and Verify

Metric	Goal / Expected Result
Order Acceptance Rate	≥ 99.9% during and after failure
Error Budget Consumption	Minimal (within SLO)
Recovery Time (RTO)	< 2–5 minutes (app)
DB Failover Time	~30–60 seconds (Aurora)
Queue Backlog	Increases, then drains to 0
Reconciliation Accuracy	100% (no duplicate or lost orders)

Results to Prove

- ✅ Traffic automatically shifts to healthy AZs.
- ✅ Aurora promotes a new writer and connections recover.
- ✅ No duplicate orders (idempotency works).
- ✅ SQS backlog is processed after recovery.
- ✅ Service meets SLOs and error budget remains intact.

Key Findings

- Temporary errors and connection resets are expected during failover.
- Retries can amplify load (retry storm) if not limited.
- Idempotency prevents duplicate orders on retries.
- Queue backlog absorbs the impact and drains safely.

Senior Engineer Takeaway

Resilience is proven, not assumed — run controlled failure tests, measure recovery, and confirm data integrity before calling it production-ready.

Project 2: Build an AWS Environment Where Stolen Credentials Cannot Easily Become a Breach

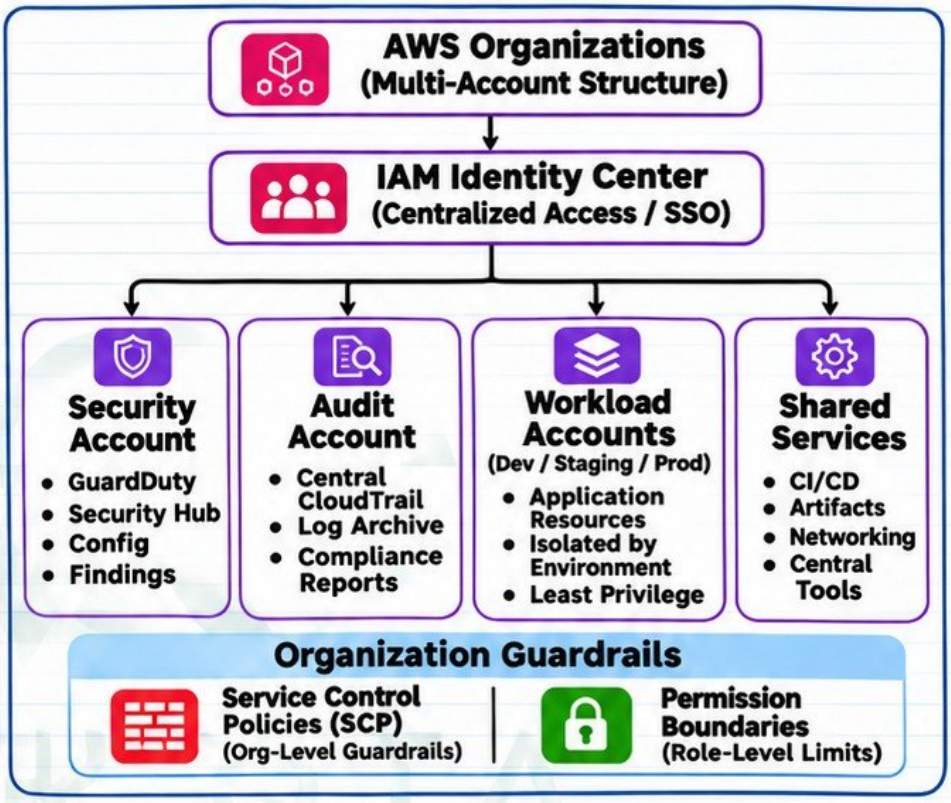
Engineer an Identity-First AWS Landing Zone

Objective

- Build a secure, multi-account AWS foundation that limits blast radius, enforces least privilege, and separates security, operations, and audit responsibilities.

Core Building Blocks

- ✅ **AWS Organizations:** Manage all accounts.
- ✅ **IAM Identity Center (SSO):** Centralized user access.
- ✅ **Service Control Policies (SCP):** Set guardrails across accounts.
- ✅ **Permission Boundaries:** Limit what roles can do.
- ✅ **AWS CloudTrail (Centralized):** Capture and store all activity.



Account Separation (Ownership)

Security	Detect, prevent, and respond to threats (GuardDuty, Security Hub).
Audit	Immutable logs and compliance evidence (CloudTrail, Log Archive).
Workload (Dev / Staging / Prod)	Run applications with isolated access and permissions.
Shared Services	Central tools (CI/CD, networking, artifact registry).

Key Security Controls

- ✅ Use SCPs to block risky actions (e.g., root access, unapproved regions).
- ✅ Apply permission boundaries to limit role privilege escalation.
- ✅ Enforce SSO with MFA for all users.
- ✅ Centralize and retain CloudTrail logs in an audit account.
- ✅ Use separate accounts to reduce blast radius from a compromised credential.

Design Considerations

- Keep security and audit accounts isolated and restricted.
- Use least privilege and role-based access.
- Apply organization-wide guardrails using SCPs.
- Enable centralized, tamper-resistant logging.

Senior Engineer Insight

A multi-account, identity-first design ensures that if one credential is stolen, the attacker is confined to a small, controlled blast radius.

Project 2: Build an AWS Environment Where Stolen Credentials Cannot Easily Become a Breach

Investigate a Compromised CI/CD Identity

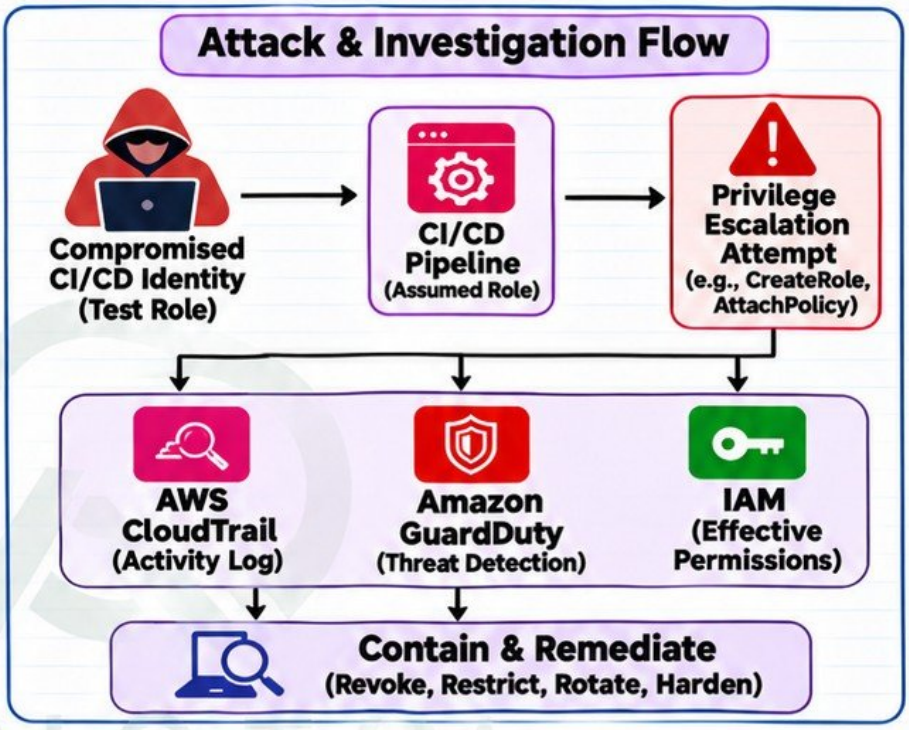
Scenario

- A CI/CD identity (e.g., GitHub Actions / CodePipeline role) was compromised and is attempting to gain higher privileges and access sensitive resources.

Investigation Goals

- ✅ Recreate the attempt in a safe, controlled environment.
- ✅ Trace activity in CloudTrail.
- ✅ Evaluate effective permissions.
- ✅ Review GuardDuty findings.
- ✅ Contain the identity quickly without impacting production.

Attack & Investigation Flow



Key Evidence to Check

CloudTrail Events	Look for sensitive API calls (e.g., CreateRole, PutPolicy, PassRole).
GuardDuty Findings	Detect anomalous IAM behavior or credential misuse.
Effective Permissions	Use IAM Policy Simulator to see what the role could actually do.
AssumeRole History	Check which role was assumed, by whom, and from where (IP/source).

Containment Steps

1. Disable or detach the compromised role or access key.
2. Revoke active sessions (STS tokens).
3. Review and fix overly permissive policies.
4. Enforce restrictive trust policies (only trusted OIDC / AWS principals).
5. Rotate credentials and validate no service disruption.

Reduce Blast Radius

- ✅ Use temporary credentials (STS) with short TTL.
- ✅ Apply permission boundaries to limit escalation.
- ✅ Enforce strict trust policies (e.g., OIDC with repo/branch conditions).
- ✅ Use SCPs to block risky actions at the organization level.

Senior Engineer Insight

The goal is not just to remove the compromised identity, but to design guardrails (STS, trust policies, boundaries, SCPs) so a single stolen credential cannot lead to a full AWS compromise.

Project 3: Stop a Healthy-Looking Deployment From Destroying Production

Build a Deployment Pipeline That Understands Business Health

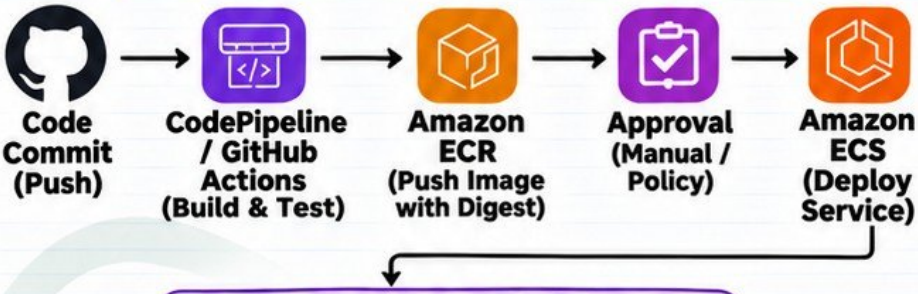
Objective

- Build a safe release pipeline that deploys only when the application and key business transactions are healthy.

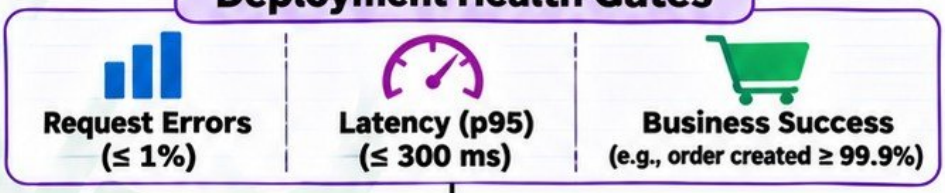
Key Components

- CodePipeline / GitHub Actions – Orchestrate release.
- Amazon ECR – Store immutable images.
- Amazon ECS – Run the service (Fargate or EC2).
- Amazon CloudWatch – Monitor health and metrics.

Deployment Pipeline (High Level)



Deployment Health Gates



Promote to Production (Only if All Gates Pass)

Release Safety Controls

Immutable Artifacts	Use image digests (sha256), not mutable tags (e.g., :latest).
Image Verification	Deploy only signed and scanned images (ECR).
Deployment Approvals	Manual or policy-based approval for production.
Health Gates	Automated checks for errors, latency, and business success.
Automatic Rollback	Revert if gates fail or metrics degrade after deployment.

Business Health Examples

- API error rate (e.g., 5xx ≤ 1%).
- p95 latency (e.g., ≤ 300 ms).
- Successful business transaction (e.g., order creation, payment, or signup ≥ 99.9%).
- No critical alarms (CPU, memory, 4xx/5xx spikes).

Why It Matters

- Prevents "healthy but broken" releases.
- Protects business data and revenue.
- Ensures only proven changes reach production.

Production Note

- Infrastructure health (e.g., HTTP 200) is not enough.
- Validate real business outcomes (e.g., orders, payments).
- Use progressive delivery (e.g., canary or blue/green) for safer rollouts.

Senior Engineer Insight

A production-ready pipeline must validate business impact, not just infrastructure health.

Project 3: Stop a Healthy-Looking Deployment From Destroying Production

Inject a Bad Release That Passes Infrastructure Health Checks

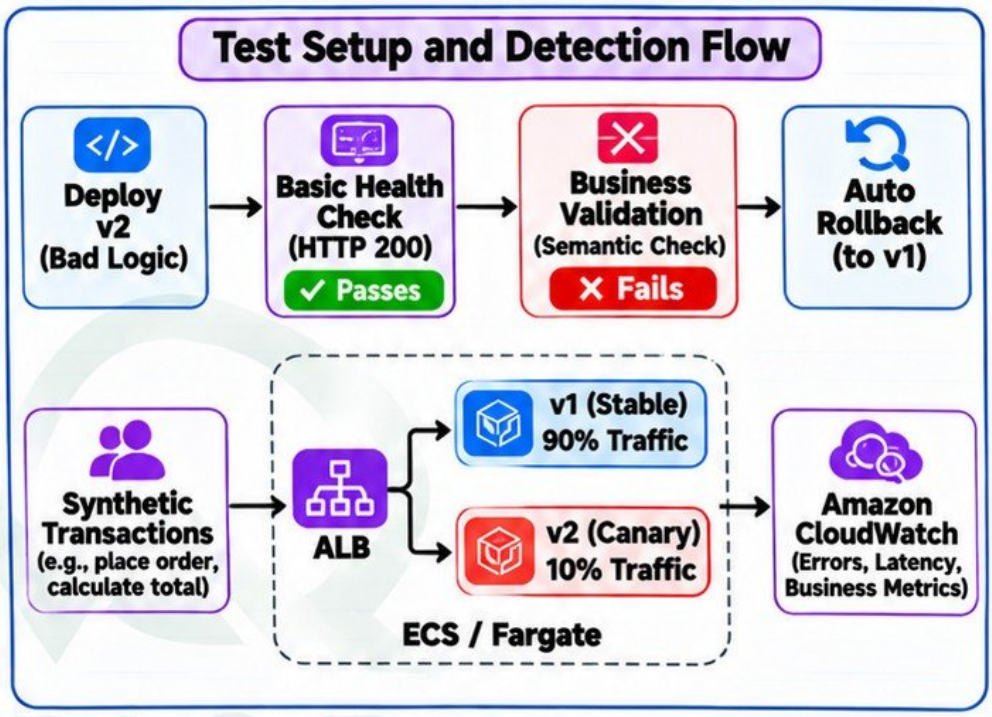
Scenario

- Deploy a new version (v2) that returns HTTP 200 but silently corrupts a business calculation (e.g., wrong discount, wrong total, or fee).

Goal

- ✅ Show why basic health checks are not enough.
- ✅ Detect the bad release with business validation.
- ✅ Use canary, synthetic transactions, and alarms.
- ✅ Automatically rollback.

Test Setup and Detection Flow



Why Basic Health Checks Fail

Checks	Only verifies app is running (e.g., HTTP 200, container healthy).
What It Misses	Business logic errors (e.g., wrong calculation, incorrect fee).
Risk	The deployment looks healthy but produces wrong business data.

Detection and Safeguards

- ✅ **Synthetic Transactions:** Run real business flows (e.g., create order, verify total).
- ✅ **Canary Comparison:** Compare v2 vs v1 results (e.g., price/total must match).
- ✅ **CloudWatch Alarms:** Trigger on business errors (e.g., ≥ 1% calculation mismatch, anomaly in totals).
- ✅ **Auto Rollback:** Revert to v1 when thresholds fail.

Why Rollback Is Not Enough

- Rollback stops further damage but does not fix data already written by the bad release.
- Corrupted records (e.g., wrong totals) still remain in the database.
- You must run a data reconciliation job to find and correct affected records.

Senior Engineer Takeaway

Infrastructure health ≠ business health. Always validate the business outcome, use canary + synthetic tests, monitor business metrics, and plan data reconciliation when a bad release occurs.

Project 4: Find the AWS Networking Failure That Looks Like an Application Bug

Build a Private Network With Observable Traffic Paths

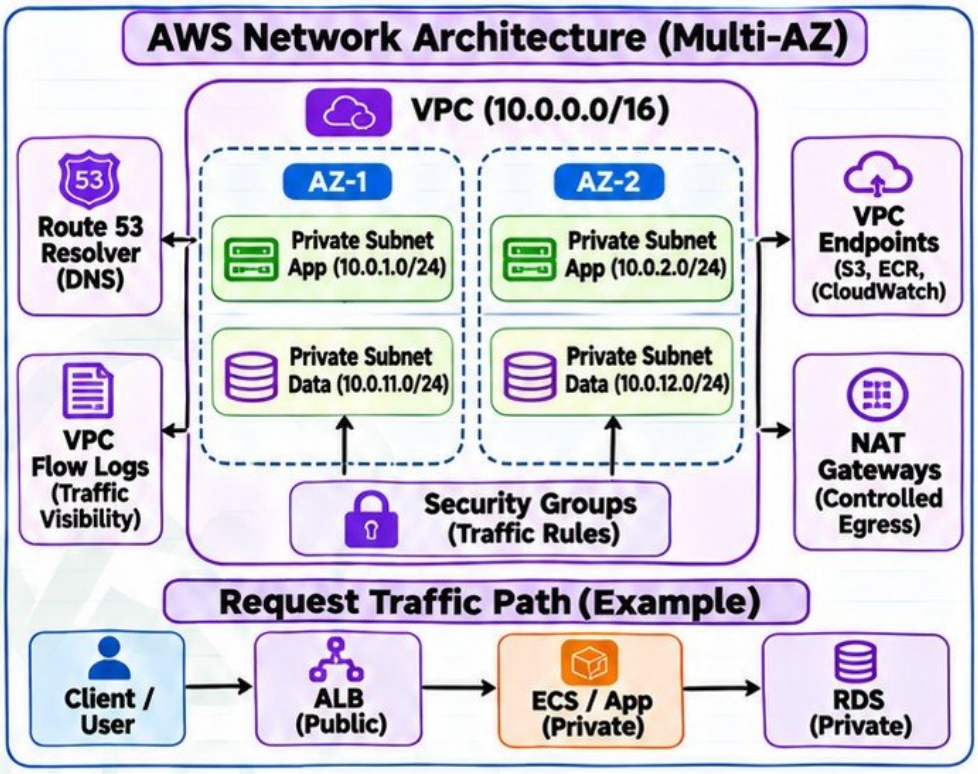
Objective

- Build a secure multi-AZ VPC with private subnets, controlled egress, VPC endpoints, DNS (Route 53 Resolver), security groups, and VPC Flow Logs to make network traffic observable and traceable.

Key Components

- Amazon VPC (multi-AZ)
- Private application subnets
- NAT Gateways (controlled egress)
- VPC Endpoints (e.g., S3, ECR, CloudWatch)
- Route 53 Resolver (DNS)
- Security Groups (traffic control)
- VPC Flow Logs (traffic visibility)

AWS Network Architecture (Multi-AZ)



Failure Types and How to Distinguish

Failure Type	Typical Symptom	How to Confirm
Routing Failure	No route / connection timeout	Check route tables, NACLs, VPC Flow Logs (REJECT).
DNS Failure	Cannot resolve hostname	Check Route 53 Resolver logs and DNS queries.
TLS Failure	TLS handshake error (e.g., cert mismatch)	Check app logs, TLS cert, and security group (443).
Application Timeout	Request times out after connection	Check app metrics, target health, and downstream latency.

What You Gain

- End-to-end visibility of network traffic.
- Faster root cause analysis (network vs app).
- Clear separation of routing, DNS, TLS, and application failures.
- Controlled and auditable egress traffic.
- Operational evidence using VPC Flow Logs and Resolver logs.

Key Takeaways

- Not all timeouts are application bugs.
- Use logs, metrics, and flow data to identify the real failure.
- A well-designed VPC makes network issues observable, reproducible, and easy to isolate.

Senior Engineer Insight

Before debugging code, prove the network path: routing, DNS, TLS, and security controls. Most "application bugs" in AWS are often networking misconfigurations.

Project 4: Find the AWS Networking Failure That Looks Like an Application Bug

Diagnose Intermittent Timeouts Without Rebooting Anything

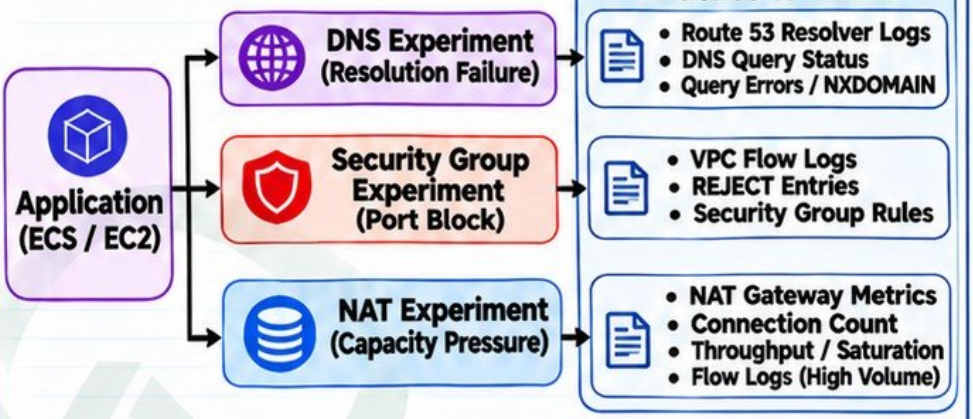
Objective

- Diagnose intermittent timeouts by running controlled experiments (DNS, Security Group, NAT) and using the right AWS evidence, not restarts.

Experiments (Controlled)

1. **DNS Failure:** Break or delay name resolution (e.g., wrong/private record).
2. **Security Group Restriction:** Block a required port (e.g., 443) for a dependency.
3. **NAT Capacity Pressure:** Simulate high outbound traffic to exhaust NAT capacity.

Experiment and Evidence Flow



Evidence Comparison and Diagnosis

Failure Domain	Key Evidence (AWS)	Observable Symptoms	Conclusion
DNS Resolution	Route 53 Resolver logs (query errors, NXDOMAIN)	Intermittent "cannot resolve host", fast failures	✓ DNS issue (name resolution failure or misconfiguration)
Security Group	VPC Flow Logs (REJECT), SG rules review	Connection timeouts, often on a specific port (e.g., 443)	✓ Traffic blocked by security group rules
NAT Capacity	NAT Gateway metrics (connections, throughput), Flow Logs (high volume)	Intermittent and rising timeouts, mostly on outbound calls	✓ NAT gateway under capacity / saturation

Tools and Metrics to Use

- ✓ Route 53 Resolver Query Logs – DNS queries, errors, and resolution status.
- ✓ VPC Flow Logs – ALLOW/REJECT, source, destination, port.
- ✓ NAT Gateway Metrics – Active connections, bytes, packets, errors.
- ✓ Application Traces – Request latency, downstream calls, and timeout points (e.g., X-Ray / OTEL).

Why Restarts Hide the Problem

- A restart temporarily clears connections, DNS cache, or resource pressure.
- It does not fix miscomfigured DNS, security rules, or NAT capacity.
- The issue will return because the root cause remains.

Key Takeaways

- Use controlled experiments to isolate the failure.
- Map each symptom to specific AWS evidence.
- Fix the root cause (DNS, SG, or NAT), not the symptoms.

Senior Engineer Insight

Always prove the failure domain with evidence (DNS, network, or capacity) before taking action. Restarts reduce symptoms, not the cause.

Project 5: Recover a Production Database After a Destructive Migration

Design a Database Recovery Strategy Around Business Data Loss

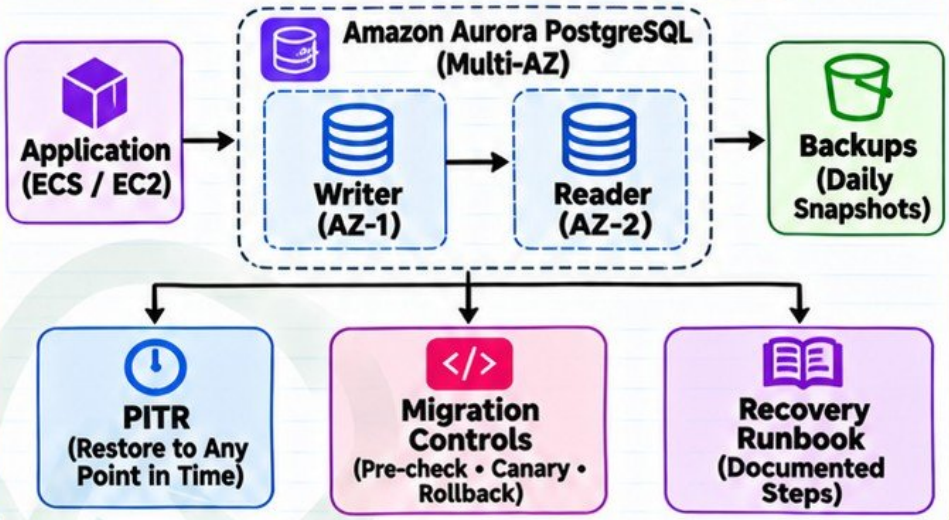
Objective

- Build a production-ready Aurora PostgreSQL setup with backups, point-in-time recovery, safe migrations, and a clear recovery runbook to restore business data after a destructive change.

Key Components

- ✅ Amazon Aurora PostgreSQL
- ✅ Automated Backups (daily + continuous logs)
- ✅ Point-in-Time Recovery (PITR)
- ✅ Migration Controls (pre-checks, canary, rollback)
- ✅ Recovery Runbook (step-by-step)

Architecture and Recovery Flow



RPO, RTO and Retention

Item	Definition	Example Target
RPO	Maximum acceptable data loss (how far back we can recover).	≤ 5 minutes (using PITR)
RTO	Maximum time to restore and get the service back.	≤ 30 minutes (restore + validate + reconnect)
Data Retention	How long backups and logs are kept.	Daily snapshots: 35 days Transaction logs: 7–35 days (align with business needs)

Recovery Runbook (High Level)

1. Detect and confirm data loss (e.g., bad migration).
2. Stop further writes (maintenance mode or read-only).
3. Restore to the target time (PITR or snapshot).
4. Validate data (counts, business transactions, and key records).
5. Reconnect the application and monitor.
6. Reconcile missing transactions (e.g., reprocess orders, payments, or updates).

High Availability vs Recoverability

High Availability (HA)	Keeps the database running during infrastructure failures (e.g., instance or AZ failure) with minimal downtime (e.g., Aurora Multi-AZ).
Recoverability	Restores data after data loss, corruption, or a destructive change (e.g., PITR, backups, rebuild and reconciliation).

Why It Matters

- Migrations can delete, overwrite, or corrupt data.
- HA alone does NOT recover lost or bad data.
- RPO/RTO define clear business expectations.
- A tested runbook reduces downtime and business risk.

Senior Engineer Insight

Design for both availability and recoverability. HA keeps the system online, while backups, PITR, and a runbook get your business data back when things go wrong.

Project 5: Recover a Production Database After a Destructive Migration

Recover From a Migration That Deletes Valid Customer Records

Objective

- Execute a destructive migration in an isolated environment, recover to a separate database, compare records, identify the missing transaction window, and reconcile affected writes.

Key Tasks

- ✅ Run a destructive migration (e.g., DELETE/incorrect script).
- ✅ Restore to a new database (using PITR or snapshot).
- ✅ Compare and find missing records.
- ✅ Reconstruct the transaction window.
- ✅ Reconcile and validate with row-level and business checks.

Experiment and Recovery Workflow

1. Baseline Data (Synthetic Customers & Orders)
2. Run Destructive Migration (e.g., DELETE valid records)
3. Recover to Separate DB (PITR / Snapshot to T−x Minutes)
4. Compare Records (Find Missing Data)
5. Reconcile and Validate (Restore and Fix Affected Writes)

Record Comparison (Example)

Metric	Before (Source)	After Migration	Recovered DB	Status
Customers (count)	10,000	9,420	10,000	✅ Restored
Orders (count)	25,000	23,800	25,000	✅ Restored
Missing Records	0	1,200	0	✅ Reconciled
Data Integrity	100%	95.2%	100%	✅ Verified

Identify the Missing Transaction Window



Verification and Integrity Checks

- ✅ Row-Level Checks: Compare counts and sample records (e.g., checksums or hashes).
- ✅ Business-Level Checks: Validate totals (e.g., orders per customer, revenue, balances).
- ✅ Transaction Reconciliation: Reapply or recreate valid writes in the missing window.
- ✅ Acceptance: Data matches source and business metrics, not just a running database.

Why "It's Running" Is Not Enough

- A started database can still have missing or corrupted data.
- You must verify record counts, transaction totals, and key business metrics.
- Success is proven by data integrity and business consistency, not by the database being online.

Key Takeaways

- ✅ Use PITR or snapshots to restore to a clean point.
- ✅ Always identify and reconcile the transaction window.
- ✅ Validate with row-level and business-level checks.
- ✅ Keep a repeatable, documented recovery process.

Senior Engineer Insight

Database recovery is complete only when the data is correct, the business records are reconciled, and integrity is verified end to end.

Project 6: Prevent a Traffic Spike From Becoming a Million-Dollar Architecture Problem

Engineer a Workload That Scales Without Scaling Its Bottlenecks

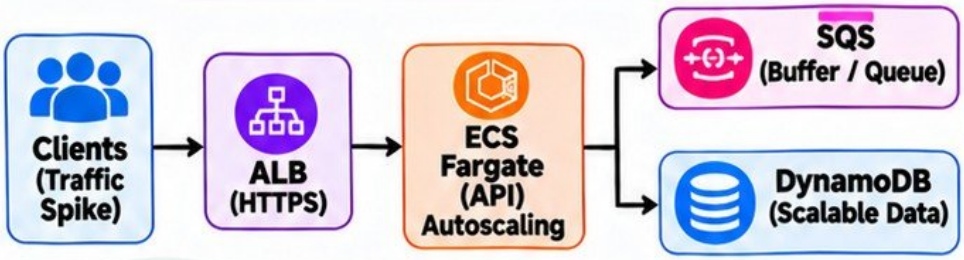
Objective

- Design a scalable API with ECS Fargate, ALB, SQS, DynamoDB, and CloudWatch. Model traffic, capacity, quotas, and cost per successful request, and avoid hidden bottlenecks.

Key Components

- ✅ Application Load Balancer (ALB) – traffic distribution.
- ✅ ECS Fargate – stateless API (autoscaling).
- ✅ Amazon SQS – buffer and decouple work.
- ✅ Amazon DynamoDB – scalable data store (on-demand or provisioned).
- ✅ CloudWatch – metrics, alarms, and cost visibility.

Architecture and Traffic Flow



Monitoring and Control



- ✅ Request rate (RPS) and concurrency
- ✅ Queue depth and age (SQS)
- ✅ DynamoDB capacity and throttles
- ✅ Service quotas and error rate
- ✅ Cost per successful request

Traffic and Capacity Model (Example)

Metric	Normal	Traffic Spike	Target / Limit
Arrival Rate (RPS)	100	1,500	Scale to ≥ 2,000
Concurrency	200	3,000	Keep p95 latency ≤ 300 ms
SQS Queue Depth	~0	25,000	Backlog < 5,000 (drain)
DynamoDB RCU/WCU	1,000	10,000	Auto scale / on-demand
Cost per Success	$0.002	$0.0035	Keep ≤ $0.004

Why More Replicas Is Not Enough

- ❌ Does not remove database contention (e.g., hot partitions, limited write capacity).
- ❌ Does not prevent unlimited queue growth (the backlog can keep rising).
- ❌ Can increase cost without increasing throughput if the bottleneck is downstream (e.g., DynamoDB or an external API).

Common Bottlenecks and Fixes

Bottleneck	Symptom	Fix
Database Contention	Higher latency, throttling	Use DynamoDB auto scaling, proper keys, and reduce hot keys.
Queue Growth	Rising backlog, delayed jobs	Set consumer scaling, DLQ, and backlog alarms.
Service Quotas	Unexpected throttling (4xx)	Monitor quotas and request increases before peak.
Cost Spike	High infra bill during spike	Set scaling limits, right-size resources, and track cost per request.

Key Takeaways

- ✅ Scale the whole flow, not just the API.
- ✅ Model traffic, concurrency, and downstream capacity before a spike.
- ✅ Use queues to absorb bursts and protect core systems.
- ✅ Watch quotas, costs, and real business metrics (e.g., successful requests, order completion).

Senior Engineer Insight

True scalability removes or manages bottlenecks end-to-end: compute, queue, database, quotas, and cost. More replicas only help if the rest of the system can keep up.

Project 6: Prevent a Traffic Spike From Becoming a Million-Dollar Architecture Problem

Break the Scaling Model With a Synthetic Traffic Surge

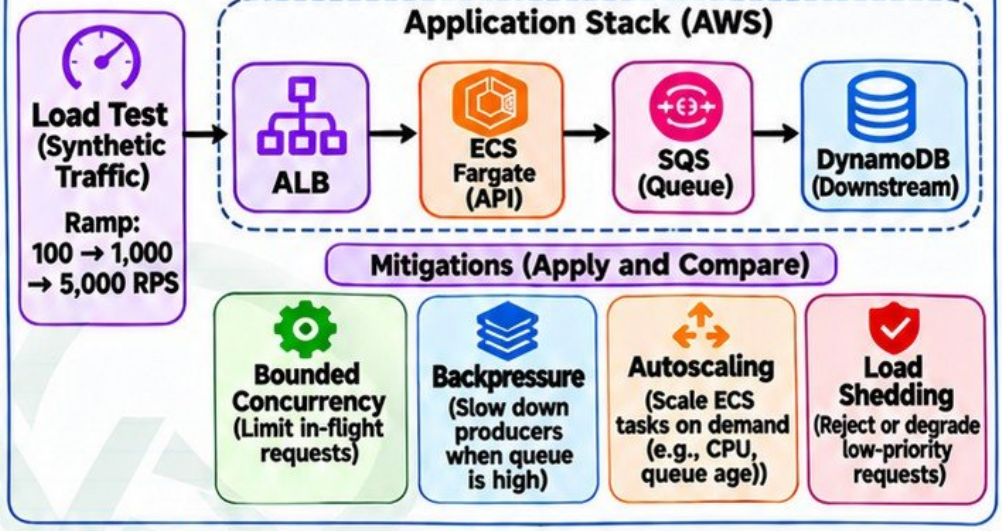
Objective

- Run a controlled load test that exceeds downstream capacity, observe queue age, throttling, p95 latency, retry amplification, and cost growth, then apply bounded concurrency, backpressure, autoscaling, and load shedding to stabilize the system and improve cost.

Key Experiments

1. Baseline load (normal traffic).
2. Traffic surge (beyond downstream capacity).
3. Apply mitigations (limits, backpressure, autoscaling, load shedding).
4. Compare performance, reliability, and unit cost.

Test Setup and Mitigation Flow



Results: Before vs After Mitigation

Metric	Baseline (100 RPS)	Surge (No Mitigation) (5,000 RPS)	After Mitigation (5,000 RPS)
p95 Latency	180 ms	4,800 ms	420 ms
SQS Queue Age	< 5 s	1,200 s (20 min)	18 s
Throttling (4xx/5xx)	0.1%	12.4%	0.6%
Retry Amplification	1.0×	3.8×	1.2×
Cost per Success	$0.0021	$0.0098 (+367%)	$0.0030 (+43%)

Why the System Broke

- Downstream (DynamoDB) capacity was the real limit.
- Queue grew without bounds (backlog and high queue age).
- Retries amplified the traffic (retry storm).
- Latency and cost increased rapidly under sustained load.

Mitigation Impact

Control	Effect	Outcome
Bounded Concurrency	Limits in-flight requests	Prevents overload
Backpressure	Slows producers when queue is high	Stabilizes queue age
Autoscaling	Adds ECS tasks on demand	Increases throughput
Load Shedding	Rejects/degrades low-priority traffic	Keeps core traffic healthy

Key Takeaways

- Scaling the API alone is not enough.
- Always model and test downstream capacity.
- Use concurrency limits and backpressure.
- Monitor queue age, p95 latency, and throttling.
- Optimize for both performance and cost (unit economics).

Senior Engineer Insight

A resilient system scales end-to-end, controls load under stress, and protects both performance and cost when traffic exceeds downstream capacity.

Project 7: Build a Kubernetes Platform That Can Survive Node Loss and Bad Scheduling

Design an EKS Platform Around Failure Domains, Not Pod Counts

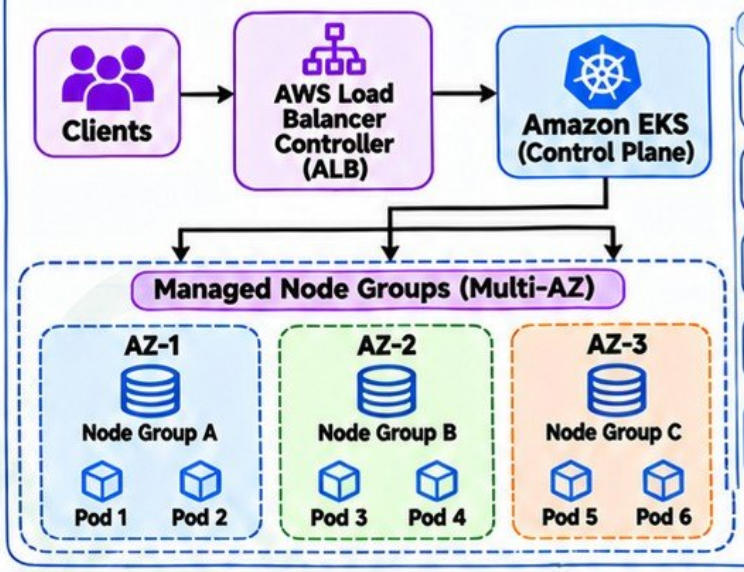
Objective

- Deploy a production-style workload on Amazon EKS with managed node groups, topology spread constraints, resource requests, PDBs, workload identity, and AWS Load Balancer Controller.

Key Components

- ✅ Amazon EKS (managed control plane).
- ✅ Managed Node Groups (multi-AZ, right sizing).
- ✅ Topology Spread Constraints (distribute across AZs/nodes).
- ✅ Resource Requests & Limits (accurate scheduling).
- ✅ PodDisruptionBudgets (PDBs) (maintain availability during changes).
- ✅ Workload Identity (IRSA) (secure AWS access).
- ✅ AWS Load Balancer Controller (ALB/NLB for ingress).

EKS Platform Architecture (High Level)



Scheduling and Resilience Controls

Control	Purpose
Resource Requests	Tell the scheduler how much CPU/ memory a pod needs.
Topology Spread	Spread pods across failure domains (e.g., AZs, nodes).
PodDisruptionBudget	Ensure a minimum number of pods stay available during drains/updates.
Autoscaling (HPA/CA)	Scale pods and/or nodes based on load.
Workload Identity (IRSA)	Secure, least-privilege access to AWS services (e.g., S3, DynamoDB).

Failure Scenario: Node Loss

1. A node in AZ-2 becomes unavailable (simulated failure).
2. Scheduler reschedules affected pods to healthy nodes in other AZs.
3. PDB keeps the required number of pods running.
4. Autoscaler adds nodes if capacity is insufficient.
5. ALB keeps traffic flowing to healthy pods without downtime.

Why Replica Counts Alone Are Not Enough

- ❌ Replicas do not control where pods run (they can end up on the same node or AZ).
- ❌ They do not prevent outages during node drains, maintenance, or AZ failure.
- ❌ They do not guarantee enough cluster capacity or proper resource allocation.

Key Takeaways

- ✅ Design for failure domains (AZs, nodes), not just pod counts.
- ✅ Use requests, PDBs, and topology spread for real resilience.
- ✅ Combine scheduling, autoscaling, and ALB for availability.
- ✅ Use workload identity (IRSA) for secure and scalable access.
- ✅ Test node loss, cordon, and drain regularly.

Senior Engineer Insight

Real Kubernetes resilience comes from thoughtful scheduling, capacity management, and disruption control, not from simply increasing replicas.

Project 7: Build a Kubernetes Platform That Can Survive Node Loss and Bad Scheduling

Trigger a Node Failure and Investigate Why Pods Stay Pending

Objective

- Simulate node unavailability and an unscheduable workload, investigate root causes (events, resources, constraints, storage, and autoscaler), and prove service continuity, recovery, and the cost trade-off.

Experiments

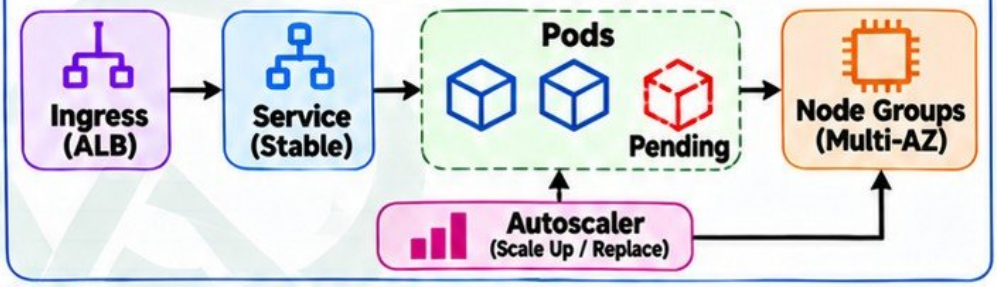
- ✅ Trigger a node failure (terminate/drain a node).
- ✅ Deploy a pod that cannot be scheduled (e.g., high CPU, wrong node selector, or missing storage).
- ✅ Investigate events, conditions, and autoscaler behavior.

Test Scenarios and Flow

Scenario 1: Node Failure
Simulate node loss (terminate or drain) and observe pod rescheduling and service continuity.

Scenario 2: Unschedulable Workload
Deploy a pod that cannot be scheduled (e.g., high resource request, topology constraint, or missing PVC).

EKS Environment (High Level)



Investigation Checklist

Area	What to Check
Node Conditions	Ready, NotReady, DiskPressure, MemoryPressure.
Events	Pod scheduling failures (e.g., Insufficient CPU, node selector mismatch, PVC not bound).
Resource Requests	CPU/memory requests vs. node capacity.
Topology Constraints	Zone/hostname spread and anti-affinity rules.
Storage Dependencies	PVC status, StorageClass, and volume availability.
Autoscaler	Scale-up decisions, limits, and why it did or did not act.

Expected Outcomes

- ✅ Pods from the failed node are rescheduled and the service remains available.
- ✅ Unschedulable pod stays Pending with clear events explaining the exact reason.
- ✅ Autoscaler adds capacity when allowed (and within limits).
- ✅ Recovery is verified and the impact on capacity and cost is measured.

Common Reasons for Pending Pods

Cause	Example	Fix
Insufficient Resources	Request > node capacity	Adjust requests/limits or add capacity.
Topology Constraint	No node in required zone	Relax or update constraints, or add nodes in that zone.
Node Selector / Affinity	No matching labels	Fix labels or selector rules.
Storage (PVC)	Volume not bound	Create/bind PVC and check StorageClass.

Trade-Off: Resilience vs. Cost

Option	Benefit	Cost Impact
Spare Capacity (Headroom)	Faster recovery, less risk of pending	Higher ongoing infrastructure cost.
On-Demand Autoscaling	Adds capacity when needed	Lower idle cost, possible delay on surge.
Tighter Resources (High Utilization)	Lower cost, higher efficiency	Higher risk of pending pods and slower recovery.

Senior Engineer Insight

Service continuity depends on correct scheduling, capacity strategy, and recovery design — not just the replica count. Resilience and cost must be balanced and intentional.

Project 8: Build an Incident Response System That Detects Real Customer Harm

Engineer Observability Around Service-Level Objectives

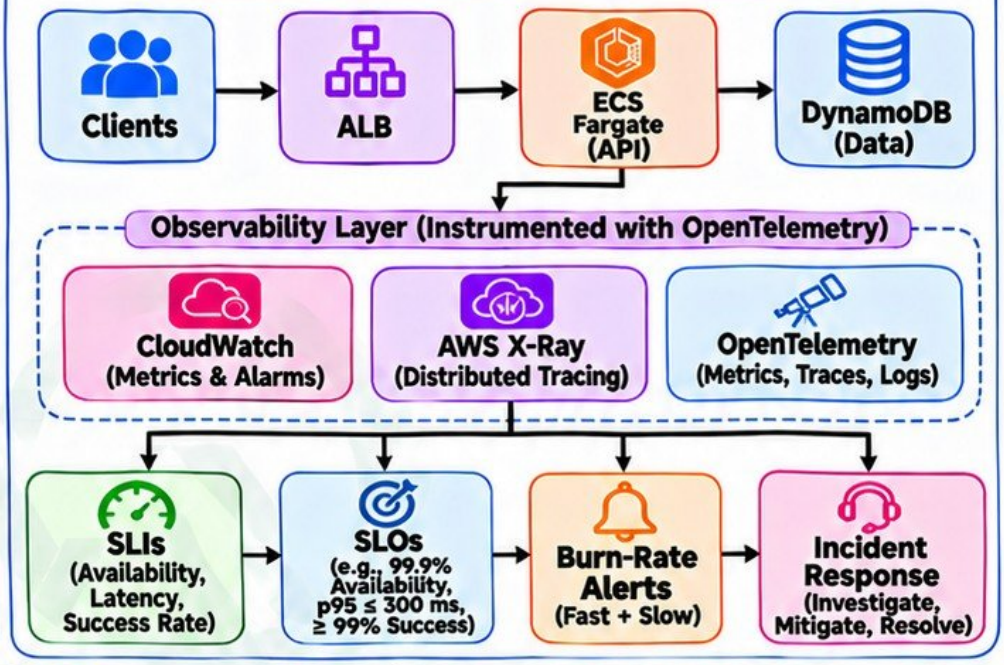
Objective

- Instrument a distributed AWS application, define meaningful SLIs and SLOs, create burn-rate alerts, correlate traces, and build dashboards that detect real customer impact and speed up incident response.

Key Components

- ✅ Amazon CloudWatch (metrics, alarms, dashboards).
- ✅ OpenTelemetry (metrics, traces, logs).
- ✅ AWS X-Ray (distributed tracing).
- ✅ SLIs and SLOs (availability, latency, business success).
- ✅ Burn-rate alerts and incident workflow (e.g., SNS, EventBridge, PagerDuty).

Architecture and Observability Flow



Example SLIs and SLOs

Area	SLI (What to Measure)	SLO (Target)
Availability	Successful requests (2xx/3xx)	≥ 99.9% (monthly)
Latency	p95 response time	≤ 300 ms
Business Operation	Successful orders / checkouts	≥ 99.0%
Error Rate	5xx / total requests	≤ 0.1%

Burn-Rate Alerting (Example)

Alert Type	Burn Rate	Purpose
Fast Burn	≥ 14×	Immediate alert for rapid SLO breach
Slow Burn	≥ 2×	Early warning for sustained risk

Symptoms vs. Causes (Examples)

Symptom (What You See)	Likely Cause (What to Investigate)
High latency (p95 ↑)	Downstream service, database contention, or network delay
Increased 5xx errors	Application error, dependency failure, or resource exhaustion
Lower business success rate	Logic error, partial outage, or external provider failure
Spikes in CPU/memory	Traffic surge, inefficient code, or missing scaling policy

Operational Outcomes

- ✅ End-to-end visibility with metrics, logs, and traces.
- ✅ SLOs that reflect real customer experience.
- ✅ Burn-rate alerts that prioritize real risk.
- ✅ Trace correlation to find the root cause quickly.
- ✅ Dashboards that separate symptoms from causes.
- ✅ A faster, evidence-based incident response.

Senior Engineer Insight

Monitoring is not just about uptime. It's about measuring and protecting what matters most: real customer outcomes and business impact.

Project 8: Build an Incident Response System That Detects Real Customer Harm

Run a Production-Style Incident With Conflicting Signals

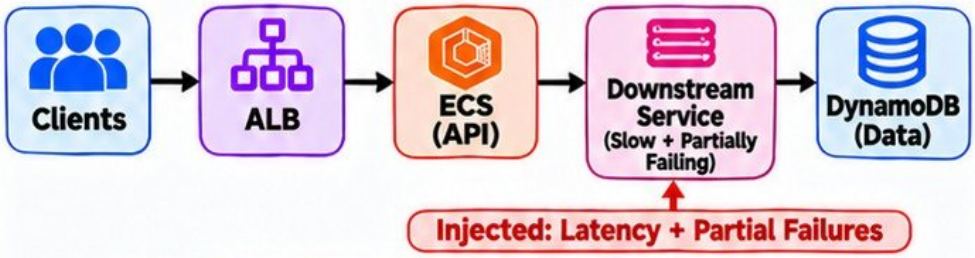
Objective

- Inject dependency latency and partial failures, analyze conflicting signals, build an incident timeline, mitigate, verify, and complete a blameless postmortem that shows why healthy infra metrics can still hide real customer impact.

Scenario Setup

1. Inject downstream latency (e.g., 2–5 s) and 10–20% request failures.
2. Keep infrastructure healthy (CPU, memory, nodes normal).
3. Observe traces, error rates, retries, and customer impact.

System and Incident Flow



What the Metrics Show

Infrastructure Metrics (Looks Healthy)

- ✓ CPU: 15–30% (Normal)
- ✓ Memory: 40–50% (Normal)
- ✓ Node health: Healthy
- ✓ Pods: Running (No restarts)
- ✓ Cluster: Stable

Customer Experience Metrics (Shows the Real Problem)

- ✗ p95 Latency: 4.8 s (↑)
- ✗ Error Rate: 12% (↑)
- ✗ Retries: 3–5× per request (↑)
- ✗ Success Rate: 88% (↓)
- ✗ Customer Impact: Timeouts & failures

Incident Timeline (Example)

Time (UTC)	Event
10:00	Normal operation (baseline).
10:05	Injected downstream latency (2–5 s).
10:08	Error rate rises, p95 latency increases.
10:12	Customer impact confirmed (timeouts).
10:15	Mitigation applied (reduce retries, increase timeout, route to healthy instance).
10:25	Service stabilized and verified.

Mitigation and Verification

Mitigation Actions	Verification Evidence
✓ Reduce retry attempts and backoff.	✓ Error rate back to < 0.5%.
✓ Increase upstream timeout (temporarily).	✓ p95 latency < 300 ms.
✓ Route traffic to a healthy downstream instance.	✓ Success rate ≥ 99.9%.
✓ Monitor error rate and p95 latency.	✓ Traces show normal downstream response time.

Key Findings

1. Infrastructure metrics can be normal while customers still experience failures.
2. Dependency latency and retries can cause partial failures and timeouts.
3. Traces and error rates reveal the real cause, not CPU or memory.
4. A clear timeline and evidence-based analysis lead to faster, safer recovery.

Blameless Postmortem (Summary)

Impact	~12% of requests failed; higher latency (~4.8 s).
Root Cause	Downstream dependency latency and partial failures.
Contributing Factors	Aggressive retries, narrow timeouts, limited fallback.
What Worked	Fast detection, trace analysis, targeted mitigation.
What to Improve	Add dependency SLOs, circuit breaker, and better alerts.
Action Items	Implement circuit breaker, tune retries, update runbook.

Senior Engineer Insight

Healthy infrastructure does not guarantee a healthy service. Always measure and respond to customer experience, not just system utilization.

Project 9: Prove That Infrastructure Changes Cannot Silently Destroy Critical Resources

Build a Terraform Delivery System With Destructive Change Controls

Objective

- Manage AWS infrastructure with Terraform, enforce change controls, prevent accidental destruction of critical resources, and deliver safe, reviewed changes across environments.

Key Components

- Terraform (IaC and modules).
- S3 remote state (versioned).
- DynamoDB state locking.
- Separate environments (dev / staging / prod).
- CI validation and policy checks.
- IAM-scoped deployment identities (least privilege).

Terraform Delivery System (End-to-End Flow)



State, Environments, and Safeguards

Remote State & Locking

S3 (Remote State) + Versioning) + DynamoDB (State Locking)

Environments (Isolated)

Dev (Fast Changes) | Staging (Validate) | Prod (Controlled)

Safeguards and Controls

Resource Protection (prevent_destroy)	Drift Detection (terraform plan)	Policy Checks (OPA / Sentinel)	IAM Least Privilege (Scoped Access)

Change Controls for Critical Resources

Control	How It Works	Protects
Deletion Guard	lifecycle,prevent_destroy = true	RDS, S3, DynamoDB
Manual Review	Required PR + plan approval	All production changes
Policy Enforcement	Block destructive actions (e.g., delete, public access)	Critical resources
Drift Detection	Compare real state vs. Terraform	Unapproved changes
Environment Isolation	Separate state and accounts	Blast radius reduction

CI/CD Pipeline (Example Flow)

1. Code Change → push Terraform changes to GitHub.
2. Validate → terraform fmt, validate, tflint.
3. Plan → generate plan and run policy checks.
4. Review → human approval (especially for prod).
5. Deploy → apply to dev → staging → prod (controlled promotion).

Common Risks and Mitigations

Risk	Impact	Mitigation
Accidental Deletion	Data or service loss	prevent_destroy, review, policy
State Corruption	Inconsistent resources	S3 versioning + DynamoDB lock
Unauthorized Change	Security and cost risk	IAM least privilege, approvals
Configuration Drift	Unexpected behavior	Regular drift detection + plan
Direct Prod Changes	High blast radius	Enforce env separation + pipeline

Expected Outcomes

- No silent or accidental destruction of critical resources.
- Auditable, reviewed, and policy-compliant changes.
- Environment isolation with controlled promotion.
- Early detection of drift or risky changes.
- A repeatable, secure, and reliable Terraform delivery process.

Senior Engineer Insight

Infrastructure changes must be intentional, reviewed, and reversible. Use automation, policy, and environment separation to make destructive changes a deliberate, controlled decision.

Project 9: Prove That Infrastructure Changes Cannot Silently Destroy Critical Resources

Catch a Terraform Change That Would Replace a Production Database

Objective

- Detect and stop a Terraform change that would replace a production database, investigate the root cause, and implement a safe migration strategy instead of applying a destructive plan.

Key Components

- Terraform plan analysis (create / update / replace).
- Immutable attribute review (e.g., engine, storage type, encryption, identifiers).
- Resource dependencies (e.g., app, network, IAM).
- Drift and state verification (terraform state, refresh).
- Policy enforcement to block destructive changes.

Scenario and Investigation Flow

Terraform (Change) → Plan (Shows Replace) → Investigate (Why Replace?) → Block (Policy/Review) → Migrate Safely (No Downtime)

Example Plan (Replacement Detected)

```
$ terraform plan
# aws_rds_instance.db will be replaced
-/+ resource "aws_rds_instance" "db" {
  ~ engine_version = "13.11" -> "14.9" (forces replacement)
  ~ storage_type   = "gp2"   -> "gp3" (forces replacement)
}
Plan: 1 to add, 0 to change, 1 to destroy.
```

Root Causes to Check

Immutable Attributes (Triggers Replacement)	Dependencies (What Will Be Affected)	Drift (Config vs Actual State)	State Behavior (State File and Refresh)	Change Intent (Accidental or Unapproved)

Safeguards and Controls

Control	Purpose
Plan Review (Required)	Catch destructive actions (replace/destroy).
Policy as Code (OPA/Sentinel)	Block high-risk changes (e.g., RDS replace).
Drift Detection (terraform plan)	Identify untracked or manual changes.
Environment Gates (dev → prod)	Require approval before production.
State Locking (S3 + DynamoDB)	Prevent concurrent or unsafe applies.

Safe Migration Strategy (Example)

1. Create a new database (e.g., RDS/Aurora) with the target configuration.
2. Replicate or migrate data (snapshot/replication/DMS).
3. Validate data and application compatibility (read/write).
4. Cut over traffic (update endpoint, tests, and monitoring).
5. Decommission the old database after a stable period.

Risks of Applying the Destructive Plan

Risk	Impact
Database Replacement	Data loss and service outage.
Downstream Breakage	Applications fail (connections, schemas).
Extended Downtime	Customer and business impact.
Recovery Uncertainty	Backups may be incomplete or slow.

Expected Outcomes

- Replacement risk detected and blocked before apply.
- Root cause explained (immutable attribute, drift, or config).
- A safe, zero/low-downtime migration path executed.
- Infrastructure changes are reviewed, controlled, and auditable.
- Production data and service availability remain intact.

Senior Engineer Insight

Never treat a destructive Terraform plan as a routine change. Understand why it replaces a resource and use a controlled migration, not a backup as a safety net.

Project 10: Design a Regional Disaster Recovery System and Prove It Actually Works

Architect a Regional Recovery Strategy With Explicit Business Trade-Offs

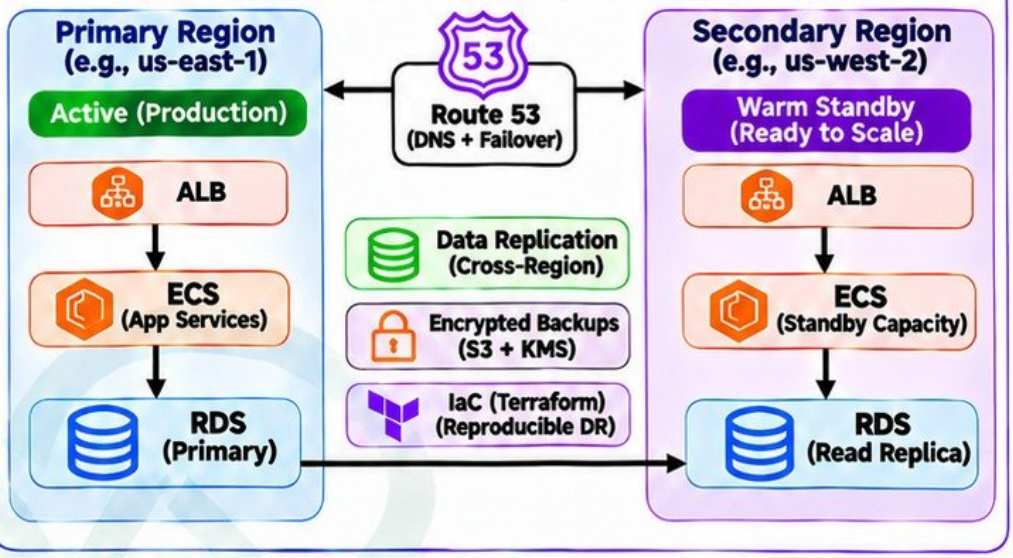
Objective

- Design a multi-region disaster recovery (DR) solution for a production-style application, using Route 53, data replication, encrypted backups, and infrastructure as code, and compare DR approaches with clear business trade-offs.

Key Components

- ✅ Amazon Route 53 (health checks + failover).
- ✅ Cross-region data replication (e.g., RDS, DynamoDB, S3 replication).
- ✅ Encrypted backups (KMS).
- ✅ Infrastructure as code (Terraform).
- ✅ Regional application capacity (standby or active-ready).

Warm-Standby DR Architecture (AWS)



DR Strategy Comparison

Approach	RTO (Target)	RPO (Target)	Data Consistency	Operational Complexity	Monthly Cost	Best Use Case
Pilot Light	Hours (2–8 h)	Minutes–Hours (15–60 min)	High	Low	$ (Low)	Cost-sensitive apps with moderate RTO.
Warm Standby	Minutes (5–30 min)	Minutes (1–5 min)	High	Medium	$$ (Medium)	Business-critical apps that need fast recovery.
Active-Active	Seconds (< 1 min)	Near-Zero (≈ 0)	Eventual or Strong (complex)	High	$$$ (High)	Mission-critical, global availability requirements.

Key Trade-Offs to Consider

Trade-Off	What It Means
RTO vs. Cost	Faster recovery requires higher standby capacity (higher cost).
RPO vs. Consistency	Lower data loss needs more frequent replication and careful design.
Simplicity vs. Resilience	Active-active gives the highest resilience but is more complex to operate.
Regional Capacity	Warm standby keeps capacity ready, so recovery is faster than pilot light.
Data Replication	Choose the right replication mode (e.g., RDS, DynamoDB, S3) for your data and consistency needs.

Expected Outcomes

- ✅ A working multi-region DR design with Route 53 failover.
- ✅ Data replicated across regions with encrypted backups.
- ✅ Infrastructure defined and reproducible with Terraform.
- ✅ A clear comparison of pilot light, warm standby, and active-active (RTO, RPO, consistency, cost).
- ✅ A justified, business-aligned DR recommendation with documented trade-offs.

Senior Engineer Insight

Disaster recovery is a business decision, not just a technical build. Choose the right strategy by balancing RTO, RPO, consistency, complexity, and ongoing cost.

Project 10: Design a Regional Disaster Recovery System and Prove It Actually Works

Conduct a Regional Disaster Recovery Exercise With Measurable Results

Objective

- Run a realistic DR exercise without disrupting AWS, execute the recovery runbook, validate data and traffic, measure RTO, RPO, cost, and capture lessons for a real regional disaster.

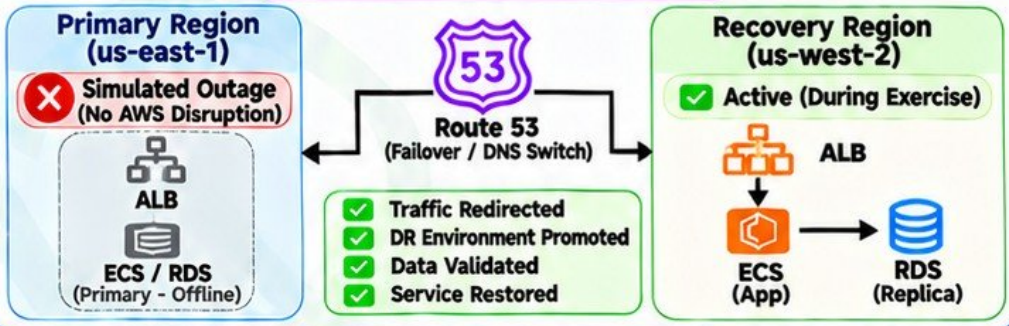
Exercise Setup

- Production-like multi-region architecture (primary + DR).
- Simulated primary-region unavailability (controlled).
- Test traffic and data validation (e.g., read/write/transactions).
- Measurable outcomes (RTO, RPO, integrity, cost).

DR Exercise Flow (No AWS Disruption)



Test DR Traffic Flow



Exercise Results (Example)

Metric	Target (SLO)	Actual (Measured)	Result
RTO (Recovery Time)	≤ 15 min	9 min 42 s	Met
RPO (Data Loss)	≤ 5 min	1 min 18 s	Met
Transaction Integrity	100%	100% (0 loss)	Met
DNS Failover Time	≤ 2 min	1 min 6 s	Met
Recovery Cost (Test)	≤ $150	$87.32	Met

Recovery Runbook (High-Level)

1. Declare DR exercise and notify stakeholders.
2. Simulate primary-region unavailability (controlled).
3. Promote recovery environment (infrastructure + app).
4. Update Route 53 to send test traffic to DR region.
5. Validate application, data, and business transactions.
6. Record results, findings, and required changes.

Failed Assumptions (Examples)

Assumption	What Actually Happened
DNS would switch instantly.	Took ~1 minute (TTL still in effect).
Replicated data is always current.	Replication lag up to ~1–2 minutes.
DR has enough capacity.	Needed to scale up to handle traffic.
Runbook is complete.	Missing steps for secret rotation and monitoring.

Changes Before a Real Disaster

- Lower Route 53 TTL for faster failover.
- Automate DR promotion steps (IaC + CI/CD runbook).
- Right-size DR capacity and test at higher load.
- Add data consistency checks and alerts.
- Update and version the runbook with lessons learned.

Senior Engineer Insight

A disaster recovery plan is not real until it is tested, measured, and improved.
The goal is predictable recovery, verified data, and a repeatable process under pressure.